

NumPy Fundamentals

A Python list of 1 million ints is a million pointers to boxed objects. A NumPy array is one contiguous block. Same maths. Not the same speed.

We run %timeit live in 8 minutes. Predict the ratio. Winner gets a V-Align sticker.

WHAT'S INSIDE

- 01** ndarray — the atom of NumPy
- 02** Vectorisation — the magic trick
- 03** Broadcasting — the part that feels like...
- 04** Vectorisation cheat sheet
- 05** Slicing, masking, indexing

WHY THIS LESSON MATTERS

80× faster — in one line of code.

NUMBER 1

1 million

From real-world deployments.

NUMBER 2

8 min

From real-world deployments.

SPEED

Now

A real reason this matters in industry today.

IMPACT

Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1

Create ndarrays of four common shapes and dtypes.

2

Replace a Python for-loop with a vectorised expression.

3

Apply broadcasting rules to different shapes.

4

Measure a speed-up with %timeit and report it.

5

Explain why NumPy underlies pandas, sklearn, PyTorch.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

ndarray — the atom of NumPy

Shape + dtype + one contiguous memory block

02

Vectorisation — the magic...

One array-wide operation; zero Python loops

03

Broadcasting — the part...

Right-align shapes; each dim must be equal or 1

04

Vectorisation cheat sheet

Every loop on a numeric array is a candidate for vectorisation.

01

PART 1 OF 4

ndarray — the atom of NumPy

Shape + dtype + one contiguous memory block

CONCEPT 1 OF 4

ndarray — the atom of NumPy

`np.array([1,2,3])` — from any iterable.

`np.zeros((3,4))` / `np.ones((3,4))` — of any shape.

`np.arange(0,10,0.5)` — NumPy's range.

`np.linspace(0,1,11)` — n evenly spaced points.

ALWAYS CHECK

`shape · ndim · size · dtype`

Four attributes catch 90% of bugs before they happen.

DEFINITION

ndarray — the atom of NumPy

Shape + dtype + one contiguous memory block

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

Vectorisation — the magic trick

`a + 5` applies to every element. No for-loop.

`a * b` multiplies element-wise.

`np.sqrt(a)` applies to every element.

The loop runs in C; Python never sees it.

EXPECT

80×–120× faster

On a million-element array, vectorised expressions beat Python loops by two orders of magnitude.

KEY POINTS

- `a + 5` applies to every element. No for-loop
- `a * b` multiplies element-wise
- `np.sqrt(a)` applies to every element
- The loop runs in C; Python never sees it

CONCEPT 3 OF 4

Broadcasting — the part that feels like magic

$(3,4) + (4,) \rightarrow$ broadcasts to $(3,4)$.

$(3,4) + (3,1) \rightarrow$ broadcasts to $(3,4)$.

$(3,4) + (3,) \rightarrow$ ERROR. Shapes misalign.

Mental model: pad shorter shape with 1s on the LEFT; then compare dimensions right-to-left.

RULE

Equal OR 1

Two aligned dims are compatible iff equal or one is 1.

DEFINITION

Broadcasting — the part that feels like...

Right-align shapes; each dim must be equal or 1

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

Vectorisation cheat sheet

Every loop on a numeric array is a candidate for vectorisation.

KEY POINTS

- Every loop on a numeric array is a candidate for vectorisation
- Practical, named, applied to a real Indian use-case.
- Practical, named, applied to a real Indian use-case.

02

PART 2 OF 4

Engage and reflect

Pause, talk, and connect what you just learned to your own work.

✦ THINK • PAIR •
SHARE

ACTIVITY

Predict the %timeit ratio — Python loop vs NumPy

PROMPT

Apply the four concepts above to one real example from your own week.

HOW TO RUN IT

1

2

3

4

Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1

Which of your past scripts would benefit most from vectorisation?

Q2

Which broadcasting shape combination confuses you most?

Q3

One habit you will adopt: "if it's numeric, it's a NumPy array".

03

PART 3 OF 4

Knowledge check

Three short questions. One minute each. Honest answers only.

np.arange(0,10,2) returns:

A

[0,2,4,6,8]

B

[2,4,6,8,10]

C

[0,1,2,...,10]

D

[0,5]

np.arange(0,10,2) returns:



CORRECT ANSWER

[0,2,4,6,8]

WHY

arange stops BEFORE the stop value. Step of 2 from 0 → 0,2,4,6,8

Shape (3,4) + shape (4,) — result?

A

error

B

(3,4)

C

(4,4)

D

(3,1)

Shape (3,4) + shape (4,) — result?



CORRECT ANSWER

(3,4)

WHY

(4,) broadcasts as (1,4), compatible with (3,4) → result (3,4)

Why is a NumPy int64 array more memory-efficient than a Python list of ints?

A

Shorter names

B

Contiguous raw bytes vs list of boxed objects

C

NumPy compresses

D

Python is lazy

Why is a NumPy int64 array more memory-efficient than a Python list of ints?



CORRECT ANSWER

Contiguous raw bytes vs list of boxed objects

WHY

List entries are full PyObjects (headers + pointer). NumPy packs raw 8-byte ints contiguously

04

PART 4 OF 4

Apply and close

One thing to remember. One thing to ship this week.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

WORKED EXAMPLE

How this lesson plays out in one real Indian context

THE SCENARIO

`np.array([1,2,3])` — from any iterable.
`np.zeros((3,4)) / np.ones((3,4))` — of any shape.
`np.arange(0,10,0.5)` — NumPy's range.
`np.linspace(0,1,11)` — n evenly spaced points.

ALWAYS CHECK

`shape · ndim · size · dtype`

Four attributes catch 90% of bugs before they happen.

THE TAKEAWAY

"Every Python ML library — pandas, sklearn, PyTorch, Hugging Face — is built on NumPy. Learn once; use forever."

WHAT TO NOTICE

1

ndarray — the atom of NumPy

2

Vectorisation — the magic...

3

Broadcasting — the part...

4

Vectorisation cheat sheet

Mini assignment — vectorise a loop

Recreate Lesson-1 grade calculation using NumPy arrays. Run both old (loop) and new (vectorised) versions.

DELIVERABLES

- Load the data into an ndarray.
- Compute per-student average using vectorised `mean(axis=1)`.
- `%timeit` both versions on 200 and on 200,000 rows.
- Report the speed-up.

WHAT 'GOOD' LOOKS LIKE

Deliverable: Notebook with two `%timeit` cells and a 2-line comment on what you observed.

ONE THING TO REMEMBER

“

"Every Python ML library — pandas, sklearn, PyTorch, Hugging Face — is built on NumPy. Learn once; use forever."

— AI Learning Hub

END OF LESSON 2

What you can now do

Each one of these is now part of your working vocabulary.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

NEXT → Lesson 3 — pandas Essentials. Series, DataFrame, indexing.